LAPORAN SEMENTARA PERTEMUAN KE 3

Nama : Yorrys Firdausi

Kelas : TKA-6D

NIM : 234308113

A. Pendahuluan

Python termasuk bahasa pemrograman yang sangat direkomendasikan untuk pemula karena cara penulisannya sederhana dan mudah dipahami. Banyaknya komunitas dan sumber belajar juga membuat proses belajar jadi lebih mudah. Dibandingkan bahasa seperti C++ atau Java yang sering menuntut pemahaman teknis cukup rumit sejak awal, Python memberi kesempatan bagi pemula untuk fokus memahami logika dan konsep dasar pemrograman terlebih dahulu tanpa harus terbebani hal teknis yang kompleks (Belajarpython.com, 2025) (Maulana, 2024).

Menurut PYPI: *Popularity of Programming Language* (2026), Python menempati posisi pertama sebagai bahasa pemrograman paling populer. Hal ini menunjukkan bahwa Python banyak digunakan di berbagai bidang salah satunya adalah *machine learning*. Dengan adanya perkembangan teknologi, hal-hal yang semula dilakukan manusia, saat ini dapat dilakukan oleh mesin. Salah satunya adalah teknologi *computer vision* (*PYPL PopularitY of Programming Language Index*, 2026).

Computer vision adalah teknik yang digunakan komputer untuk memahami dan mengambil informasi penting dari gambar digital. Sistem ini bekerja dengan menerima gambar, lalu memprosesnya menggunakan pengolahan citra dan pengenalan pola. Selanjutnya, algoritma computer vision menganalisis gambar tersebut untuk mencapai tujuan atau tugas tertentu (Suradi & Rasyid, 2022). Dengan *computer vision*, mesin tidak hanya dapat mengidentifikasi suatu objek, namun juga dapat menganalisa suatu pergerakan objek tersebut.

Pada praktikum ini, library yang digunakan adalah *Scikit-Learn. Scikit-Learn* merupakan salah satu library pada Python yang dirancang untuk mempermudah proses pengembangan machine learning melalui antarmuka pemrograman (API) yang terstruktur dan mudah digunakan. Library ini dikembangkan oleh berbagai kontributor dari berbagai negara dan telah banyak dimanfaatkan baik di lingkungan industri maupun akademik (M. N. Fahmi, 2023). Dengan adanya Scikit-Learn, pengguna dapat membangun, melatih, dan mengevaluasi model machine learning dengan sintaks yang lebih sederhana dan sistematis.

Dari sisi performa, Scikit-Learn memiliki kinerja yang optimal karena dibangun di atas pustaka NumPy (*Numerical Python*) dan SciPy (*Scientific Python*) yang mendukung komputasi numerik secara efisien. NumPy sendiri merupakan pustaka fundamental dalam pemrograman Python, khususnya dalam manipulasi data dan perhitungan numerik. Pustaka ini menyediakan dukungan kuat untuk operasi aljabar linear, pengolahan matriks, serta komputasi pada array multidimensi yang menjadi dasar dalam berbagai aplikasi ilmiah dan teknik. Melalui struktur data yang efisien dan algoritma yang telah dioptimalkan, NumPy mampu memproses data dalam jumlah besar dengan waktu komputasi yang relatif cepat (N. S. Fahmi & Asih, 2025). Dukungan inilah yang membuat proses perhitungan dalam Scikit-Learn menjadi lebih stabil dan efisien. Meskipun demikian, Scikit-Learn tetap memiliki keterbatasan ketika digunakan untuk pengolahan dataset berskala sangat besar.

B. Tujuan

Berdasarkan uraian latar belakang yang telah dijelaskan sebelumnya mengenai penggunaan Python, Scikit-Learn, serta penerapan computer vision dalam sistem pengenalan gesture, maka praktikum ini memiliki beberapa tujuan sebagai berikut:

1. Mengembangkan sistem deteksi tangan menggunakan MediaPipe dan OpenCV sebagai dasar pengambilan data citra secara *real-time*.
2. Mengimplementasikan algoritma machine learning menggunakan Scikit-Learn untuk melakukan klasifikasi gesture tangan atau huruf alfabet SIBI.

3. Mengekstraksi dan mengolah landmark tangan menjadi fitur numerik yang dapat digunakan dalam proses pelatihan dan pengujian model.
4. Menguji kinerja serta akurasi sistem dalam mengenali gesture atau huruf yang ditampilkan melalui kamera.
5. Menganalisis hasil prediksi sistem sebagai bentuk evaluasi terhadap performa model yang telah dibuat.

C. Manfaat

Adapun manfaat praktikum ini adalah sebagai berikut:

1. Meningkatkan pemahaman mahasiswa mengenai penerapan Python dalam pengolahan citra dan pengembangan sistem *computer vision*.
2. Memberikan pengalaman langsung dalam mengimplementasikan MediaPipe dan OpenCV untuk mendeteksi serta mengekstraksi landmark tangan secara *real-time*.
3. Melatih kemampuan dalam membangun dan menguji model klasifikasi menggunakan *Scikit-Learn* pada kasus pengenalan gesture atau huruf alfabet SIBI.
4. Mengembangkan keterampilan analisis terhadap performa dan akurasi sistem yang telah dibuat.
5. Menambah wawasan mengenai pemanfaatan teknologi machine learning dalam bidang komunikasi, khususnya pada sistem penerjemah bahasa isyarat.

D. Kode Program Beserta Penjelasannya

1. Pengumpulan Data

```
import os
import cv2
```

- Program diawali dengan mengimpor library os dan OpenCV (cv2). Library os digunakan untuk mengelola direktori penyimpanan data, sedangkan OpenCV digunakan untuk mengakses kamera dan mengambil gambar dari webcam.

```
DATA_DIR = r'D:\Yorrys\Kontrol Cerdas\Minggu 3\DATA CADANGAN'

if not os.path.exists(DATA_DIR):
    os.mkdir(DATA_DIR)
```

- Bagian ini menentukan lokasi folder utama sebagai tempat penyimpanan dataset.
- Program melakukan pengecekan apakah folder tersebut sudah tersedia.
- Jika folder belum ada, sistem secara otomatis membuat folder baru agar data dapat tersimpan dengan terstruktur.

```
number_of_classes = 3
dataset_size = 100
```

- Variabel ini digunakan untuk menentukan jumlah kelas gesture yang akan direkam.
- dataset_size menunjukkan jumlah gambar yang akan diambil pada setiap kelas.
- Dengan pengaturan ini, setiap kelas akan memiliki jumlah data yang sama sehingga dataset menjadi seimbang.

```
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Kamera tidak bisa dibuka")
    exit()
```

- Program mengaktifkan kamera utama (indeks 0) sebagai sumber input gambar.
- Sistem melakukan pengecekan untuk memastikan kamera berhasil diakses.
- Jika kamera tidak dapat dibuka, program akan dihentikan untuk mencegah terjadinya error.

```
for j in range(number_of_classes):

    class_dir = os.path.join(DATA_DIR, str(j))
    if not os.path.exists(class_dir):
        os.mkdir(class_dir)

    print(f'Collecting data for class {j}')
```

- Program melakukan perulangan sesuai jumlah kelas yang telah ditentukan.
- Untuk setiap kelas, sistem membuat folder khusus berdasarkan indeks kelas.
- Informasi kelas yang sedang direkam ditampilkan pada terminal sebagai penanda proses pengambilan data.

```
while True:
    ret, frame = cap.read()

    if not ret:
        print("Gagal membaca kamera")
        break

    cv2.putText(frame, 'Ready? Press "Q"!', (100, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0),
3)

    cv2.imshow('frame', frame)

    if cv2.waitKey(15) == ord('q'):
        break
```

- Program menampilkan tampilan kamera secara langsung sebelum proses pengambilan data dimulai.
- Sistem memberikan instruksi kepada pengguna untuk bersiap melalui teks yang ditampilkan pada layar.
- Pengguna diminta menekan tombol tertentu sebagai tanda bahwa proses perekaman dapat dimulai.
- Jika kamera gagal membaca frame, proses akan dihentikan.

```
counter = 0
while counter < dataset_size:
    ret, frame = cap.read()

    if not ret:
        print("Gagal membaca kamera")
        break

    cv2.imshow('frame', frame)
    cv2.waitKey(10)
```

```
    cv2.imwrite(os.path.join(class_dir, f'{counter}.jpg'),
frame)
    counter += 1
```

- Setelah proses dimulai, sistem mengambil gambar secara berulang hingga jumlah dataset yang ditentukan tercapai.
- Setiap gambar yang ditangkap langsung disimpan ke dalam folder kelas yang sesuai.
- Penamaan file dilakukan secara berurutan agar data tersusun rapi dan mudah digunakan saat proses training model.

```
cap.release()
cv2.destroyAllWindows()
```

- Setelah seluruh proses pengambilan data selesai, program menonaktifkan kamera.
- Seluruh jendela tampilan ditutup untuk memastikan sistem berhenti dengan baik dan tidak ada proses yang masih berjalan.

2. Ekstrasi Data

Beberapa baris kode yang sudah dijelaskan pada poin sebelumnya akan dilewati pada penjelasan kode program berikut.

```
import os
import pickle
import mediapipe as mp
import cv2
```

- Program diawali dengan mengimpor beberapa library yang dibutuhkan, yaitu os untuk membaca struktur folder dataset, pickle untuk menyimpan data dalam bentuk file biner, MediaPipe untuk mendeteksi landmark tangan, serta OpenCV untuk membaca gambar.

```
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    static_image_mode=True,
    max_num_hands=1
)
```

- Bagian ini menginisialisasi modul Hands dari MediaPipe.

- Parameter static_image_mode=True menunjukkan bahwa sistem bekerja pada gambar statis (bukan video real-time).
- max_num_hands=1 membatasi deteksi hanya pada satu tangan dalam setiap gambar.

```
DATA_DIR = r'D:\Yorrys\Kontrol Cerdas\Minggu 3\DATA SIBI'

data = []
labels = []
```

- Program menentukan lokasi folder dataset yang berisi subfolder tiap kelas huruf atau gesture.
- Variabel data digunakan untuk menyimpan fitur numerik hasil ekstraksi landmark.
- Variabel labels digunakan untuk menyimpan label kelas dari masing-masing data.

```
for dir_ in os.listdir(DATA_DIR):

    class_path = os.path.join(DATA_DIR, dir_)

    if not os.path.isdir(class_path):
        continue
```

- Program membaca seluruh folder kelas yang terdapat pada direktori dataset.
- Setiap folder dianggap sebagai satu kelas (misalnya huruf A, B, C, dan seterusnya).
- Jika terdapat file selain folder, program akan melewatinya agar tidak terjadi kesalahan pembacaan.

```
for img_path in os.listdir(class_path):

    data_aux = []
    x_ = []
    y_ = []
```

- Program melakukan perulangan untuk membaca seluruh gambar dalam setiap folder kelas.
- data_aux digunakan untuk menyimpan fitur koordinat yang telah dinormalisasi.
- x_ dan y_ digunakan untuk menampung nilai koordinat asli sebelum dilakukan normalisasi.

```
img = cv2.imread(os.path.join(class_path, img_path))

if img is None:
    continue
```

- Gambar dibaca menggunakan OpenCV dari folder kelas yang sesuai.
- Jika gambar gagal dibaca, program akan melewati file tersebut untuk mencegah error pada proses selanjutnya.

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        results = hands.process(img_rgb)

        if results.multi_hand_landmarks:

            for hand_landmarks in
results.multi_hand_landmarks:

                for landmark in hand_landmarks.landmark:
                    x_.append(landmark.x)
                    y_.append(landmark.y)

                for landmark in hand_landmarks.landmark:
                    data_aux.append(landmark.x - min(x_))
                    data_aux.append(landmark.y - min(y_))

            data.append(data_aux)
            labels.append(dir_)

# Simpan pickle SETELAH semua data selesai diproses
save_path = r'D:\Yorrys\Kontrol Cerdas\Minggu 3\DATA
SIBI\data.pickle'

with open(save_path, 'wb') as f:
    pickle.dump({'data': data, 'labels': labels}, f)

print("Data berhasil disimpan!")
```

- Gambar dikonversi dari format BGR ke RGB agar sesuai dengan format yang digunakan oleh MediaPipe, kemudian sistem melakukan proses deteksi landmark tangan pada gambar tersebut.

- Jika landmark tangan berhasil terdeteksi, program mengambil seluruh koordinat x dan y dari setiap titik landmark yang tersedia.
- Koordinat tersebut kemudian dinormalisasi dengan cara mengurangi setiap nilai terhadap nilai minimum pada sumbu x dan y, sehingga fitur yang dihasilkan tidak bergantung pada posisi tangan di dalam gambar.
- Hasil normalisasi disimpan sebagai fitur numerik (data_aux), kemudian dimasukkan ke dalam variabel data beserta label kelasnya sebagai target klasifikasi.
- Setelah seluruh gambar selesai diproses, data dan label disimpan ke dalam file berformat pickle agar dapat digunakan pada tahap pelatihan model machine learning.
- Program menampilkan pesan sebagai indikator bahwa proses penyimpanan dataset telah berhasil dilakukan.

3. Pelatihan & Pengujian

Beberapa baris kode yang sudah dijelaskan pada poin sebelumnya akan dilewati pada penjelasan kode program berikut.

```
import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
```

- Program mengimpor library yang dibutuhkan untuk proses pelatihan model.
- RandomForestClassifier digunakan sebagai algoritma klasifikasi.
- train_test_split digunakan untuk membagi data menjadi data latih dan data uji.
- accuracy_score digunakan untuk menghitung tingkat akurasi model.
- NumPy digunakan untuk mengelola data dalam bentuk array numerik.

```
data_dict = pickle.load(open('D:\Yorrys\Kontrol
Cerdas\Minggu 3\DATA SIBI\data.pickle', 'rb'))
```

```
data = np.array(data_dict['data'])
labels = np.array(data_dict['labels'])
```

- Program memuat dataset yang sebelumnya telah disimpan dalam file pickle.
- Data fitur dan label dipisahkan dari dictionary yang telah disimpan.
- Keduanya kemudian dikonversi ke dalam bentuk array NumPy agar dapat diproses oleh algoritma machine learning.

```
x_train, x_test, y_train, y_test = train_test_split(data,
labels, test_size = 0.1, shuffle = True, stratify = labels)
```

- Dataset dibagi menjadi data latih dan data uji.
- Sebanyak 10% data digunakan sebagai data pengujian, sedangkan sisanya digunakan untuk pelatihan.
- shuffle=True bertujuan untuk mengacak data sebelum dibagi agar distribusi lebih merata.
- stratify=labels memastikan proporsi setiap kelas tetap seimbang pada data latih dan data uji.

```
model = RandomForestClassifier()
model.fit(x_train, y_train)
```

- Model Random Forest diinisialisasi tanpa parameter tambahan (menggunakan pengaturan default).
- Model kemudian dilatih menggunakan data latih untuk mempelajari pola dari fitur landmark tangan terhadap masing-masing kelas gesture.

```
y_pred = model.predict(x_test)
score = accuracy_score(y_pred, y_test)
print('{}% of samples were classified correctly
!'.format(score*100))
```

- Setelah proses pelatihan selesai, model digunakan untuk memprediksi data uji.
- Hasil prediksi dibandingkan dengan label asli untuk menghitung tingkat akurasi.

- Nilai akurasi ditampilkan dalam bentuk persentase sebagai indikator performa model dalam mengenali gesture atau huruf SIBI.

```
f = open('D:\Yorrys\Kontrol Cerdas\Minggu 3\DATA
SIBI\model.p', 'wb')
pickle.dump({'model' : model}, f)
f.close()
```

- Model yang telah dilatih kemudian disimpan kembali dalam bentuk file pickle.
- Penyimpanan ini bertujuan agar model dapat digunakan kembali pada tahap implementasi sistem deteksi secara real-time tanpa perlu melakukan pelatihan ulang.

4. Implementasi

Beberapa baris kode yang sudah dijelaskan pada poin sebelumnya akan dilewati pada penjelasan kode program berikut.

```
import pickle
import cv2
import mediapipe as mp
import numpy as np
```

- Program menggunakan library pickle untuk memuat model yang telah dilatih sebelumnya, OpenCV untuk mengakses kamera dan menampilkan video, MediaPipe untuk mendeteksi landmark tangan, serta NumPy untuk mengelola data numerik hasil ekstraksi fitur.

```
model_dict = pickle.load(open(r'D:\Yorrys\Kontrol
Cerdas\Minggu 3\DATA SIBI\model.p', 'rb'))
model = model_dict['model']
```

- Bagian ini memuat model hasil pelatihan yang telah disimpan dalam file pickle.
- Model tersebut nantinya digunakan untuk melakukan prediksi huruf berdasarkan fitur landmark tangan yang terdeteksi secara real-time.

```
hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
```

```
        min_tracking_confidence=0.7
    )
```

- Sistem menginisialisasi MediaPipe Hands dalam mode video (real-time).
- Deteksi dibatasi hanya untuk satu tangan agar proses lebih stabil dan fokus.
- Nilai confidence diatur cukup tinggi (0.7) untuk meningkatkan ketepatan deteksi dan mengurangi kesalahan pembacaan landmark.

```
labels_dict = {
    0: 'A', 1: 'B', 2: 'C', 3: 'D', 4: 'E',
    5: 'F', 6: 'G', 7: 'H', 8: 'I', 9: 'J',
    10: 'K', 11: 'L', 12: 'M', 13: 'N', 14: 'O',
    15: 'P', 16: 'Q', 17: 'R', 18: 'S', 19: 'T',
    20: 'U', 21: 'V', 22: 'W', 23: 'X', 24: 'Y',
    25: 'Z'
```

- Dictionary ini berfungsi sebagai pemetaan antara hasil prediksi model (berupa angka) dengan huruf alfabet SIBI.
- Dengan pemetaan ini, hasil klasifikasi dapat langsung ditampilkan dalam bentuk huruf yang mudah dipahami pengguna.

```
for landmark in hand_landmarks.landmark:
    x_.append(landmark.x)
    y_.append(landmark.y)

for landmark in hand_landmarks.landmark:
    data_aux.append(landmark.x - min(x_))
    data_aux.append(landmark.y - min(y_))
```

- Program mengambil seluruh koordinat landmark tangan yang terdeteksi.
- Koordinat tersebut kemudian dinormalisasi agar fitur tidak bergantung pada posisi tangan dalam frame, melainkan pada bentuk gesturenya.
- Hasil normalisasi inilah yang menjadi input bagi model klasifikasi.

```
if len(data_aux) == 42:  # 21 titik x,y
    prediction = model.predict(np.array([data_aux]))
    predicted_letter = labels_dict[int(prediction[0])]
```

- Sistem memastikan jumlah fitur sesuai dengan data saat training (21 titik × 2 koordinat = 42 fitur).

- Model kemudian melakukan prediksi huruf berdasarkan fitur yang diberikan.
- Hasil prediksi dikonversi menjadi huruf menggunakan dictionary label.

```
cv2.putText(frame,
                        predicted_letter,
                        (50, 80),
                        cv2.FONT_HERSHEY_SIMPLEX,
                        2,
                        (0, 0, 255),
                        4)

cv2.imshow('Deteksi Huruf SIBI', frame)
```

- Huruf hasil prediksi ditampilkan pada layar secara real-time.
- Video hasil deteksi ditampilkan dalam jendela sehingga pengguna dapat melihat landmark tangan sekaligus hasil klasifikasi.

E. Analisis

Secara keseluruhan, ketiga program yang telah dibuat menunjukkan tahapan pengembangan sistem pengenalan huruf SIBI berbasis MediaPipe Hands, OpenCV, dan *Machine Learning* secara bertahap dan terstruktur. Program pertama berfokus pada proses pengumpulan data (*data acquisition*), yaitu mendeteksi landmark tangan menggunakan MediaPipe dan mengekstraksi koordinat setiap titik tangan. Pada tahap ini, sistem tidak langsung melakukan klasifikasi, melainkan membangun dataset terlebih dahulu dengan cara menormalisasi koordinat landmark agar tidak bergantung pada posisi tangan di dalam frame. Hasil ekstraksi tersebut kemudian disimpan dalam bentuk file pickle sebagai dataset training. Tahap ini dapat dikatakan sebagai fondasi utama, karena kualitas data sangat menentukan performa model pada tahap berikutnya.

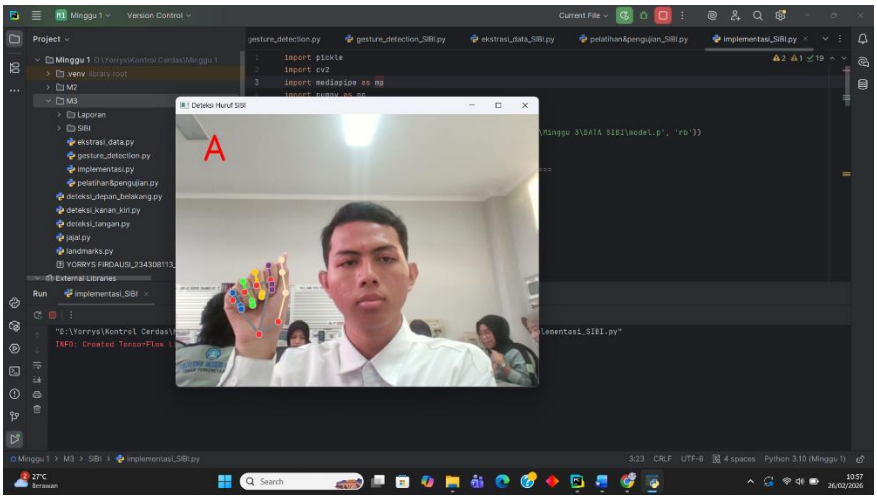
Program kedua merupakan tahap pengolahan dan pelatihan model (*training phase*). Dataset yang sebelumnya telah dikumpulkan kemudian dibagi menjadi data latih dan data uji menggunakan metode train-test split. Model klasifikasi yang digunakan adalah RandomForestClassifier dari library *scikit-learn*. Pada tahap ini, sistem sudah masuk ke proses pembelajaran pola, di mana model mempelajari karakteristik koordinat landmark untuk setiap huruf. Setelah

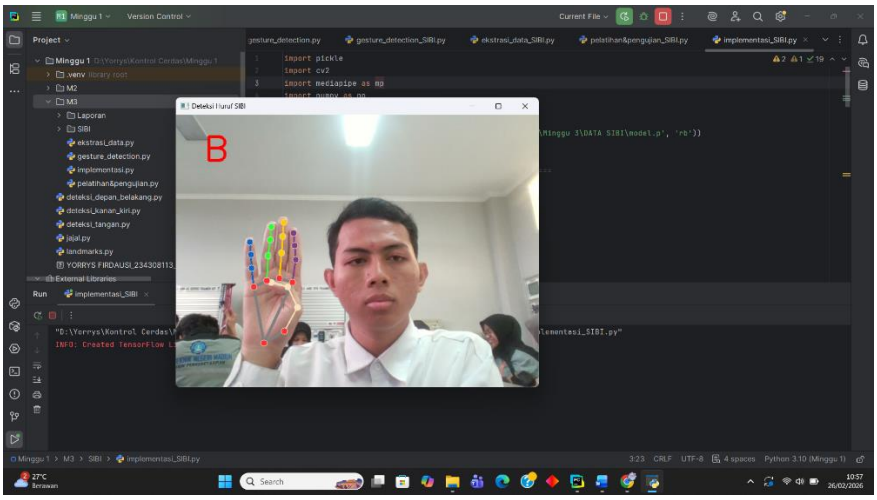
pelatihan selesai, akurasi dihitung untuk mengetahui seberapa baik model mengenali data uji. Model yang telah terlatih kemudian disimpan agar dapat digunakan kembali tanpa perlu melakukan training ulang. Tahap ini menunjukkan transisi dari sekadar pengumpulan data menjadi sistem yang memiliki kemampuan prediksi.

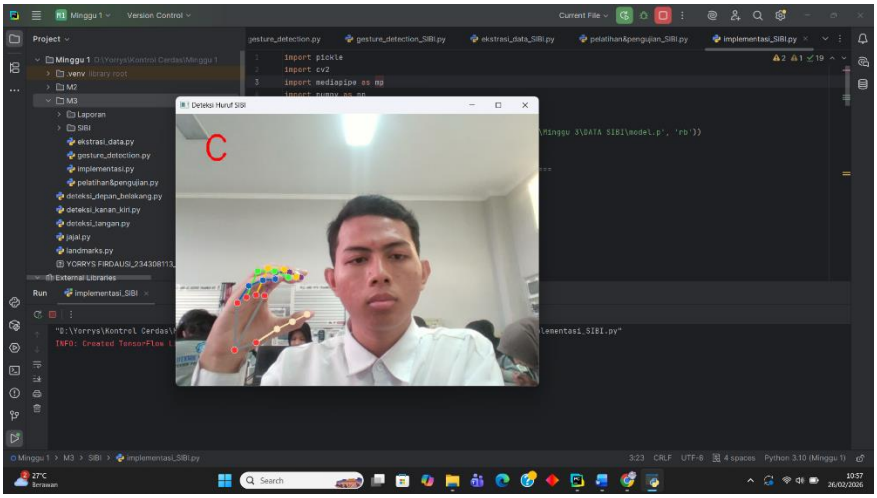
Program ketiga merupakan implementasi real-time yang lebih aplikatif. Model yang telah dilatih sebelumnya dimuat kembali, lalu digunakan untuk melakukan prediksi huruf berdasarkan input langsung dari webcam. Sistem mendeteksi tangan secara *real-time*, menormalisasi koordinat landmark seperti pada tahap training, lalu memasukkan data tersebut ke model untuk menghasilkan prediksi huruf A–Z. Hasil prediksi kemudian ditampilkan langsung pada layar menggunakan OpenCV. Pada tahap ini, sistem sudah menjadi sebuah aplikasi utuh yang mampu mendeteksi, memproses, menganalisis, dan menampilkan hasil klasifikasi secara langsung.

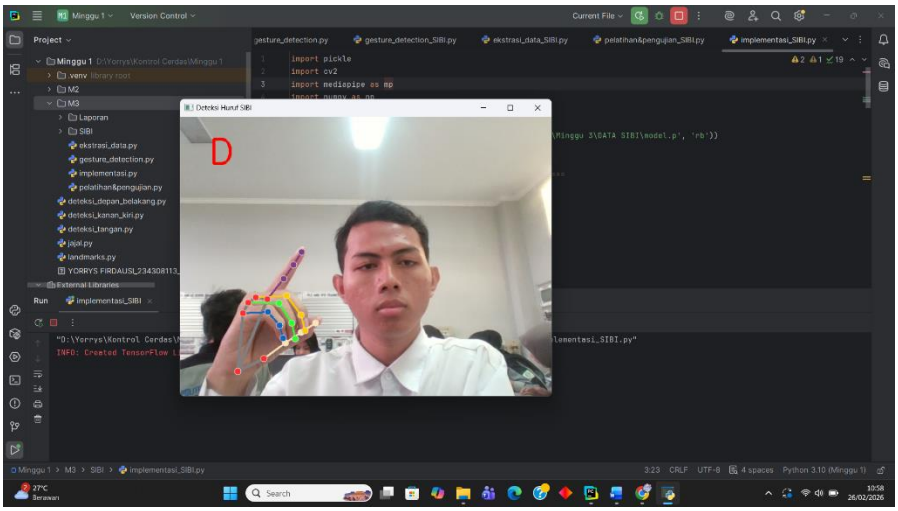
Inti dari keseluruhan program tersebut adalah menunjukkan alur lengkap pembangunan sistem kecerdasan buatan sederhana, mulai dari pengumpulan data, pelatihan model, hingga implementasi real-time. Awalnya sistem hanya mengekstraksi koordinat landmark tangan, kemudian belajar mengenali pola huruf melalui proses training, dan akhirnya mampu mengambil keputusan berupa prediksi huruf secara langsung dari kamera. Dengan kata lain, rangkaian program ini tidak hanya melatih penggunaan library, tetapi juga membangun pemahaman mengenai alur kerja sistem machine learning secara menyeluruh: data, model, prediksi, aplikasi.

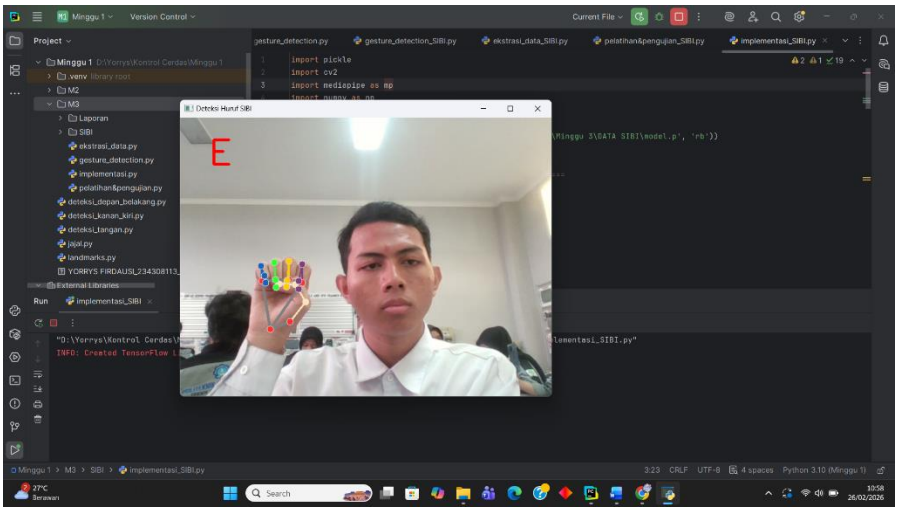
F. Hasil

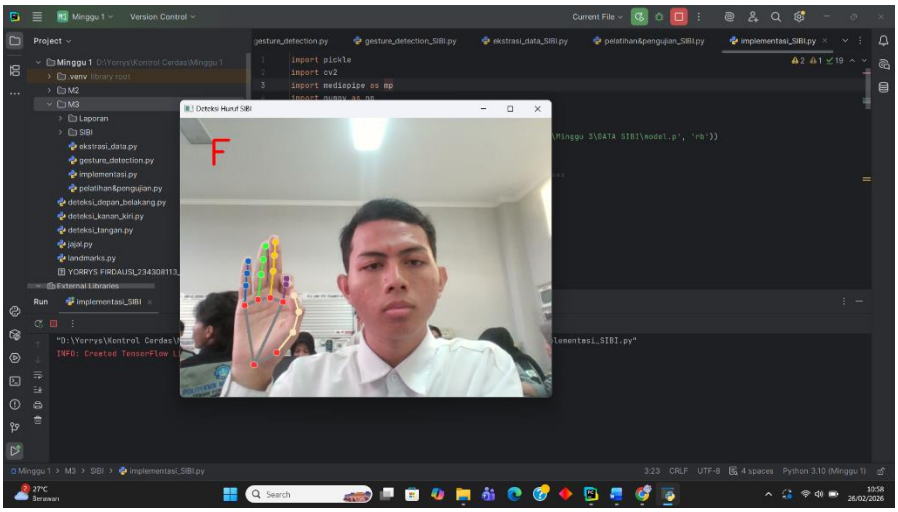

Deteksi Huruf SIBI
A

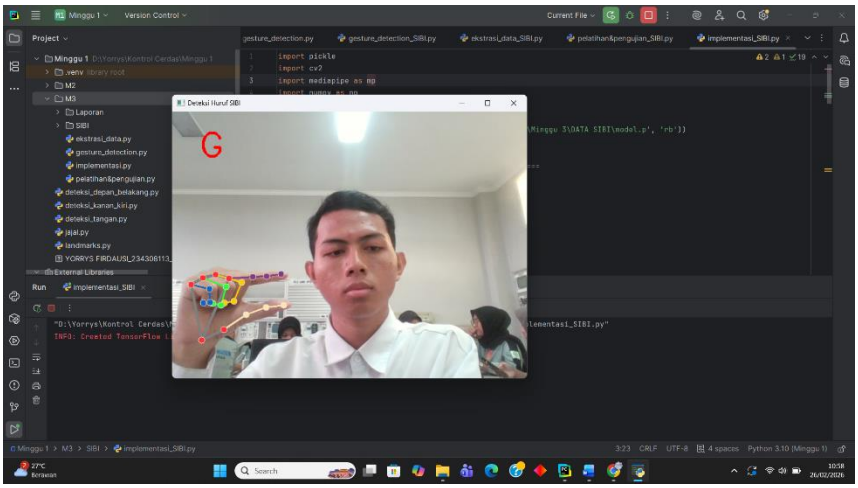

Deteksi Huruf SIBI
B

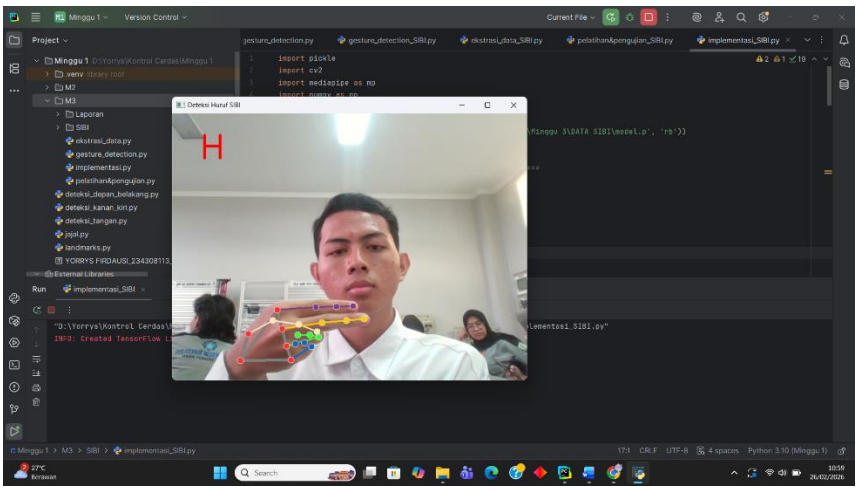

Deteksi Huruf SIBI
C

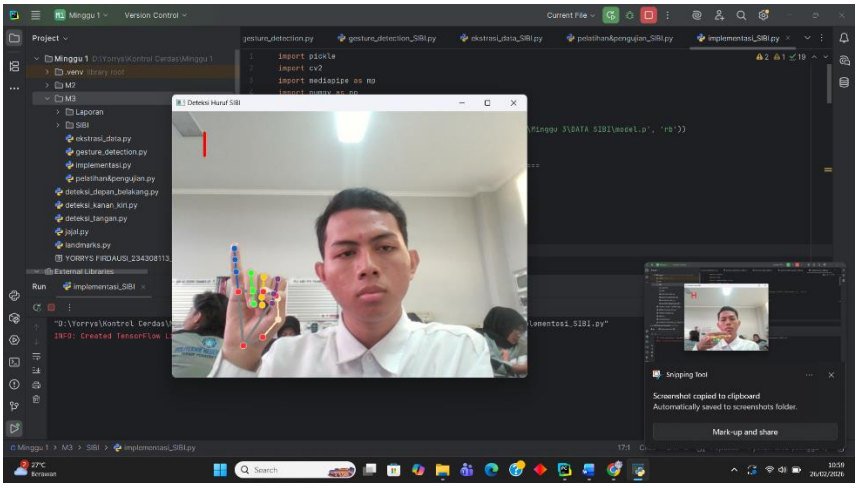

Deteksi Huruf SIBI
D

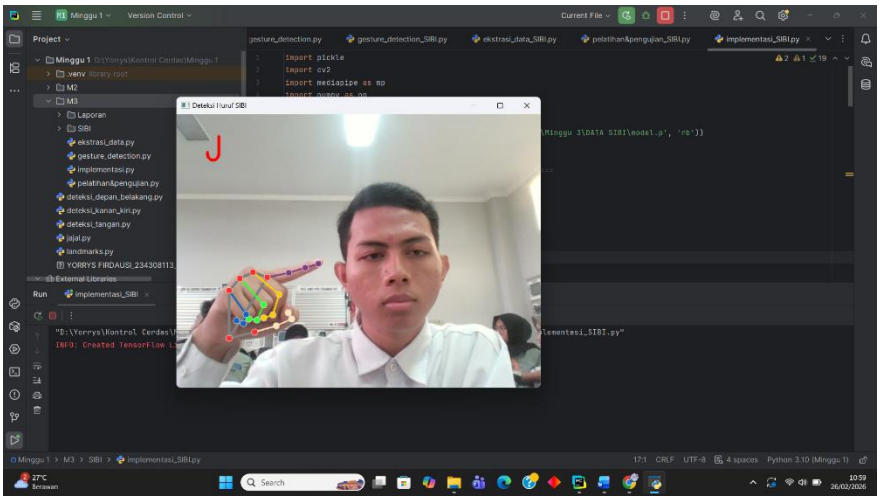

Deteksi Huruf SIBI
E

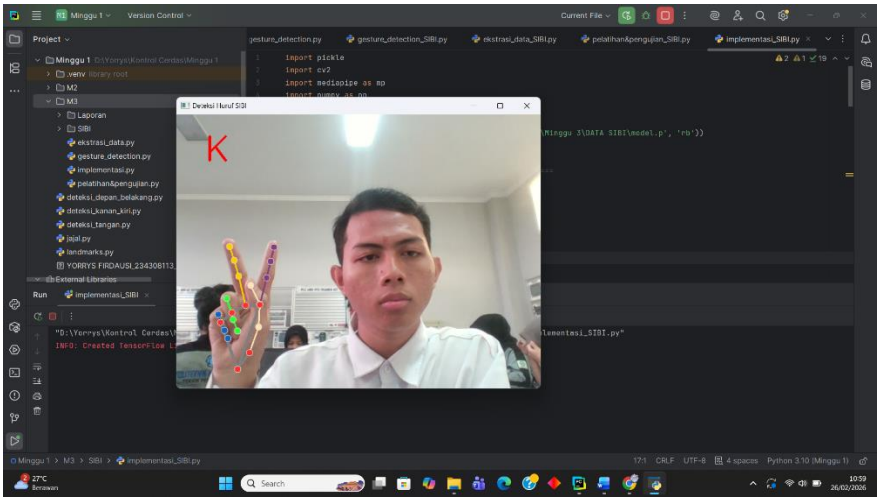

Deteksi Huruf SIBI
F

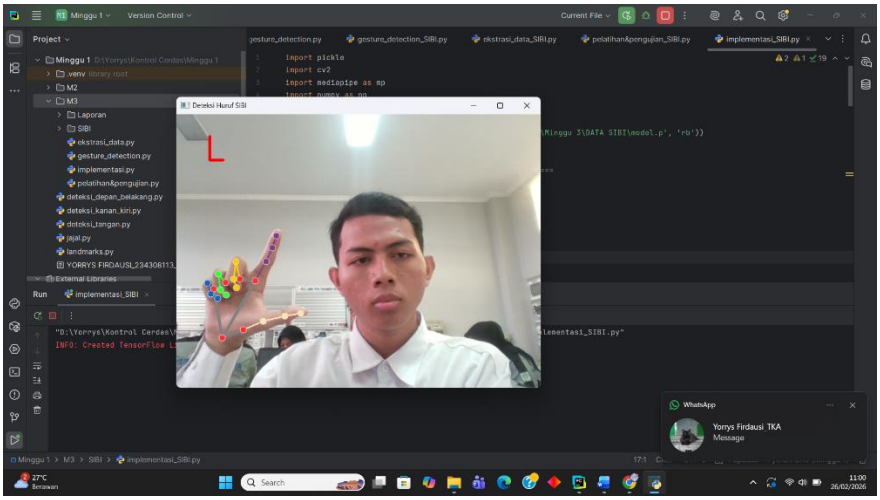

Deteksi Huruf SIBI
G

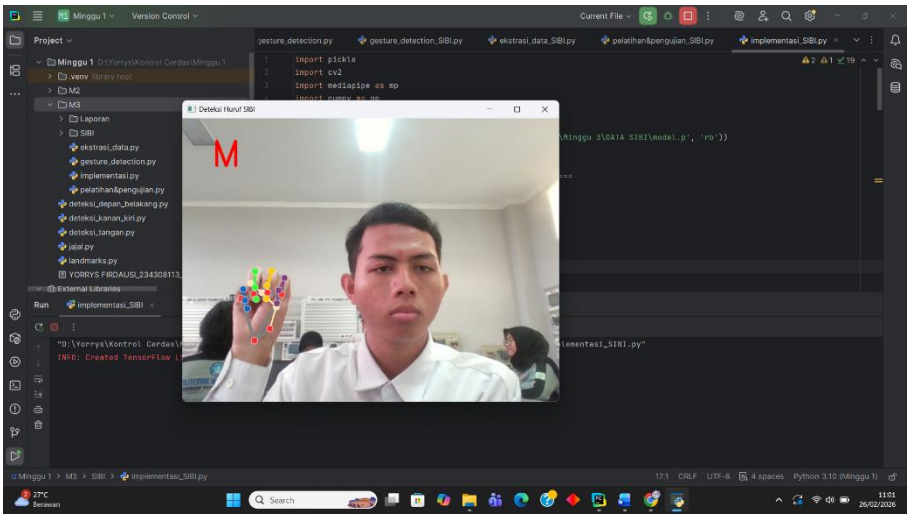

Deteksi Huruf SIBI
H

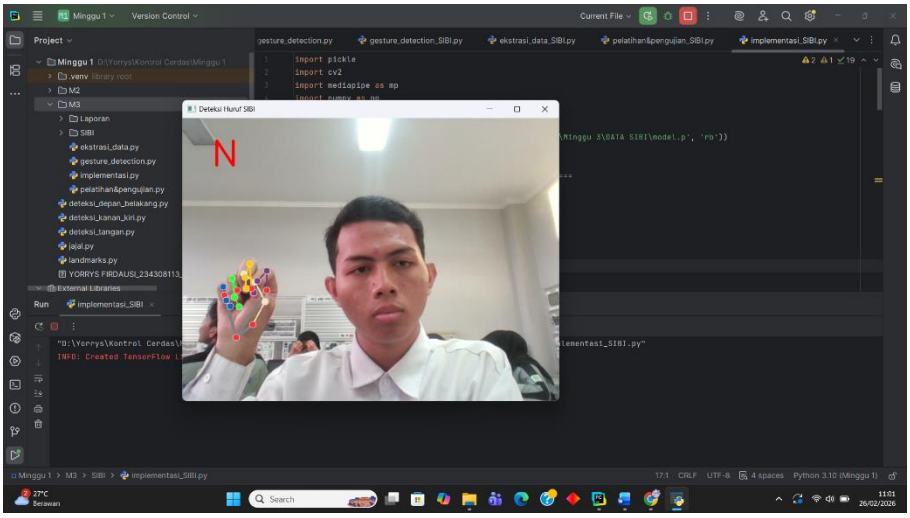

Deteksi Huruf SIBI
I
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

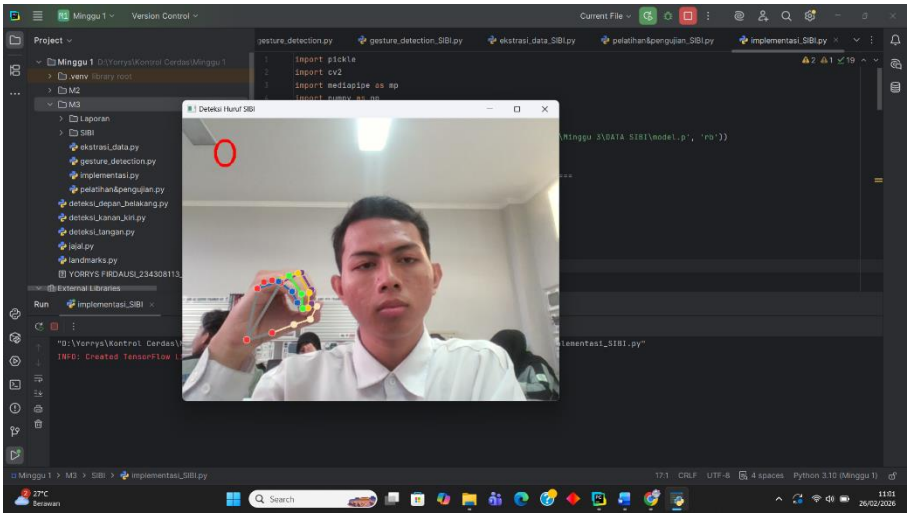

Deteksi Huruf SIBI
J

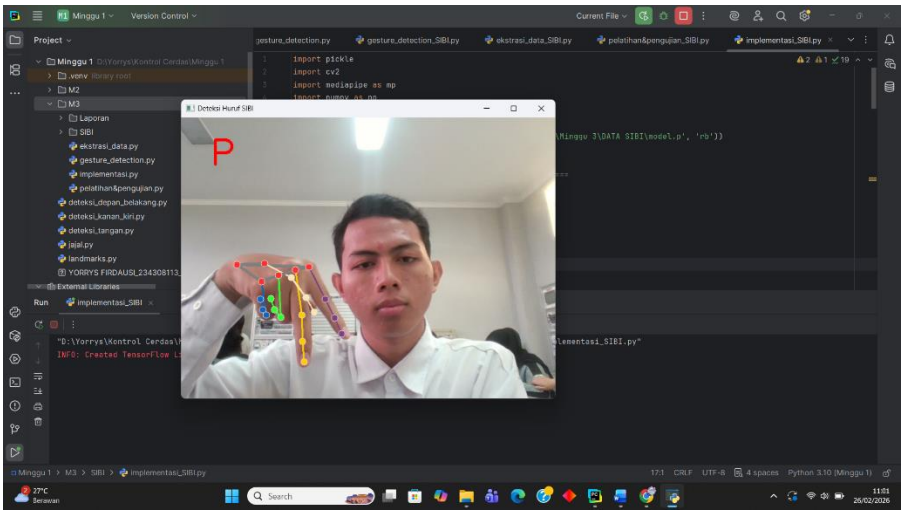

Deteksi Huruf SIBI
K

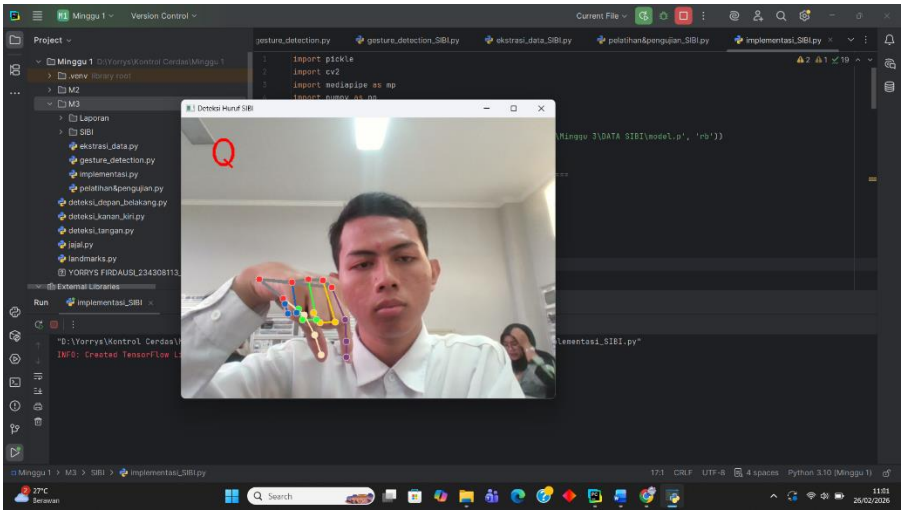

Deteksi Huruf SIBI
L

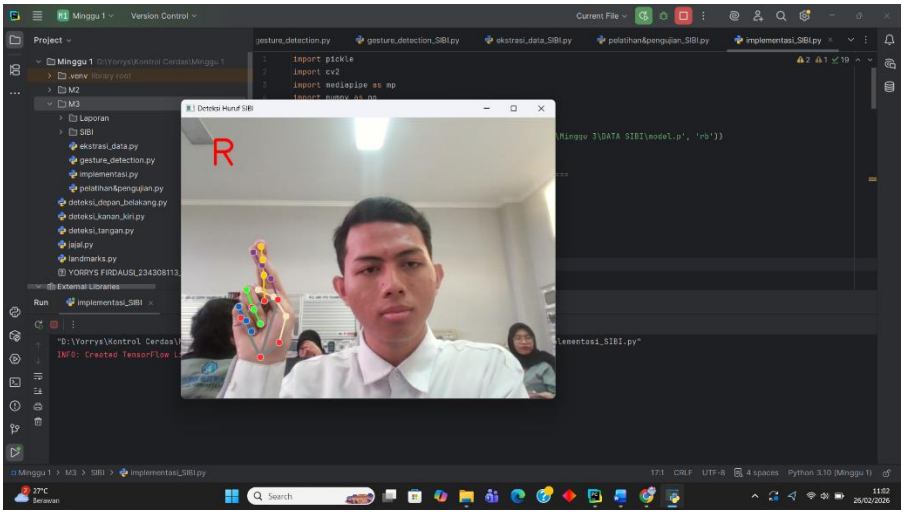

Deteksi Huruf SIBI
M

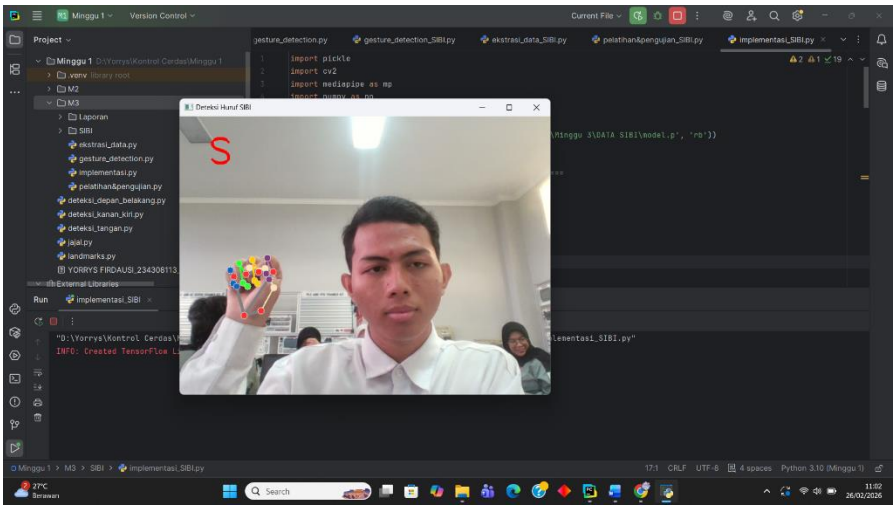

Deteksi Huruf SIBI
N

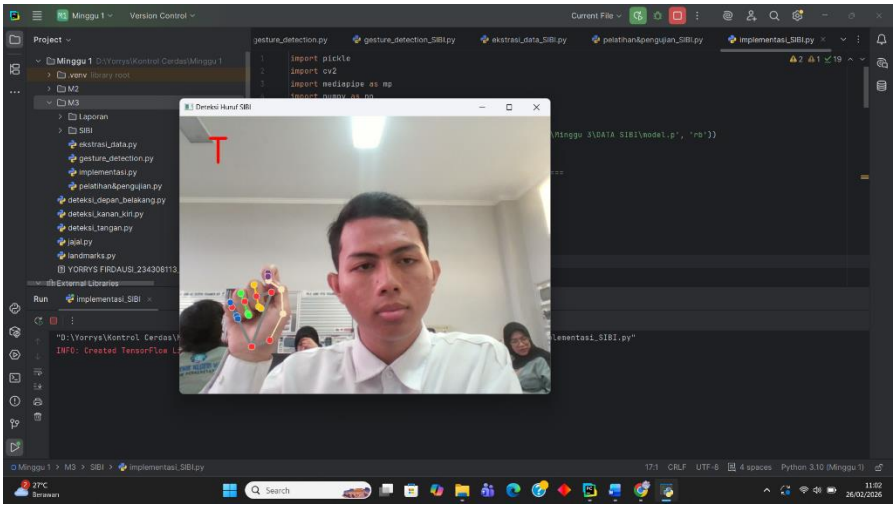

Deteksi Huruf SIBI
O

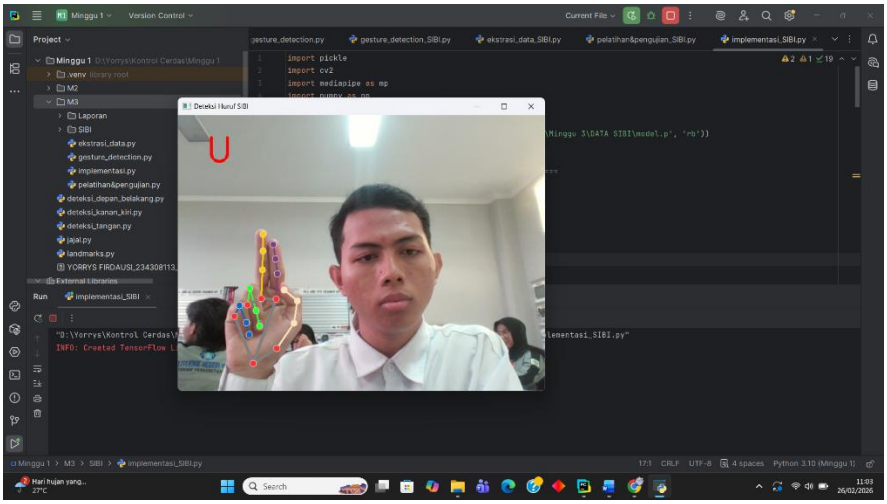

Deteksi Huruf SIBI
P

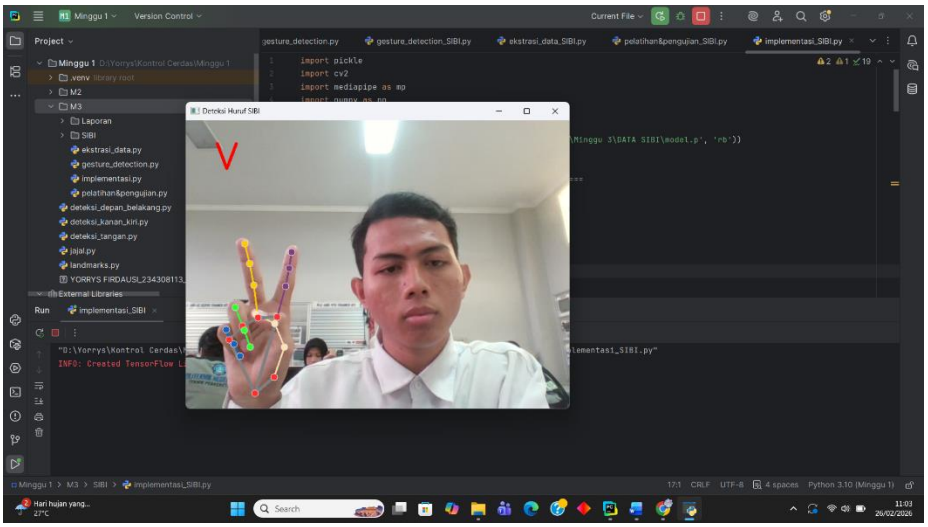

Deteksi Huruf SIBI
Q

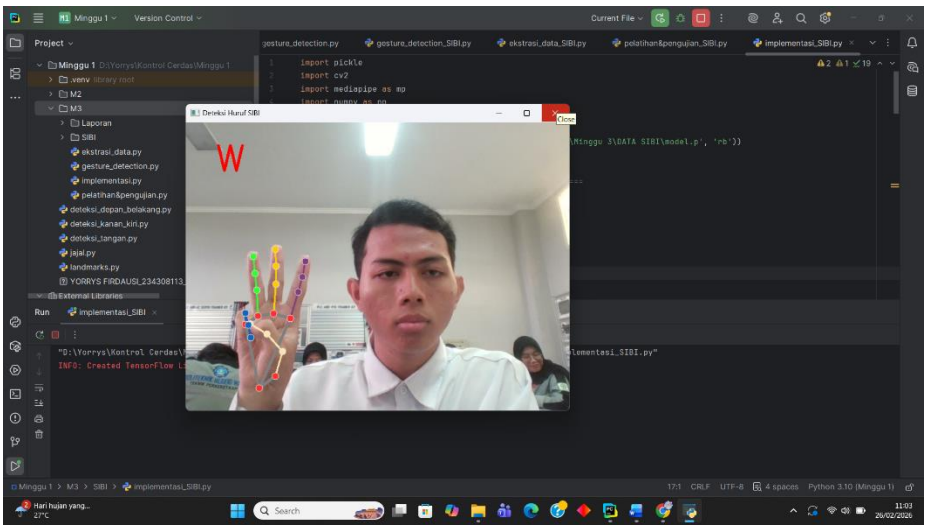

Deteksi Huruf SIBI
R

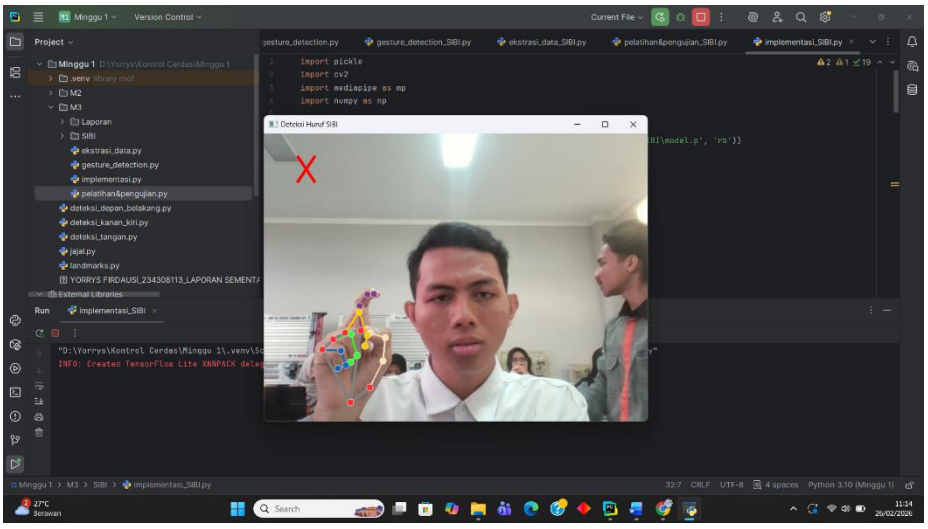

Deteksi Huruf SIBI
S

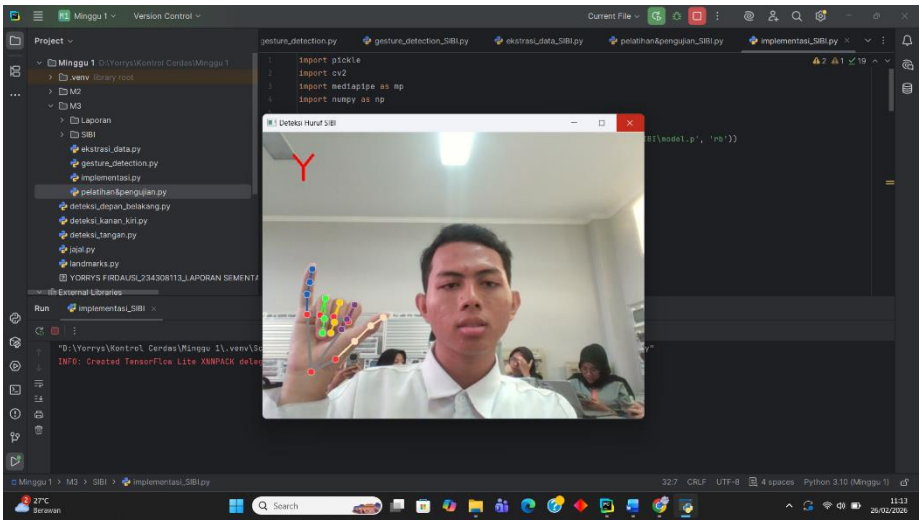

Deteksi Huruf SIBI
T

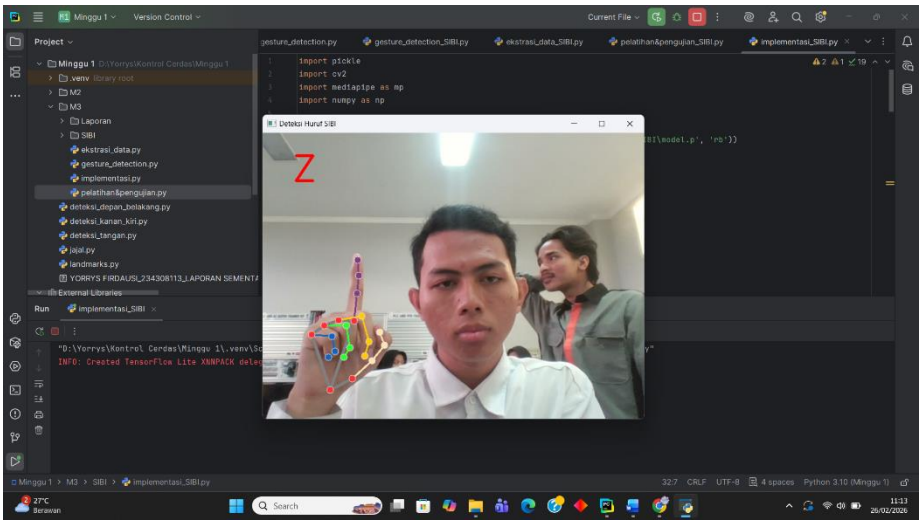

Deteksi Huruf SIBI
U


Deteksi Huruf SIBI
V


Deteksi Huruf SIBI
W


Deteksi Huruf SIBI
X





Gambar 1 - 26. Hasil Percobaan Pembacaan dari A-Z

G. Kesimpulan

Setelah melakukan praktikum dan menganalisisnya, dapat diperoleh beberapa kesimpulan sebagai berikut :

1. Praktikum ini membuktikan bahwa sistem pengenalan huruf SIBI dapat dibangun dengan memanfaatkan deteksi landmark tangan dari MediaPipe yang dipadukan dengan OpenCV sebagai pengolah citra secara *real-time* melalui *webcam*.
2. Proses pengumpulan dan normalisasi koordinat 21 titik landmark tangan menghasilkan dataset yang terstruktur, sehingga data dapat digunakan sebagai dasar pelatihan model klasifikasi secara lebih akurat dan konsisten.

3. Penerapan algoritma Random Forest pada tahap training menunjukkan bahwa model machine learning mampu mempelajari pola dari data koordinat tangan dan melakukan prediksi huruf dengan tingkat akurasi yang baik.
4. Tahapan pengembangan dilakukan secara sistematis, dimulai dari akuisisi data, pelatihan model, hingga implementasi prediksi secara *real-time*, sehingga alur kerja sistem kecerdasan buatan dapat dipahami secara menyeluruh dan terstruktur.
5. Secara umum, praktikum ini tidak hanya meningkatkan pemahaman dalam penggunaan library computer vision dan machine learning, tetapi juga melatih kemampuan analisis dalam mengubah data hasil deteksi menjadi sistem klasifikasi yang fungsional dan aplikatif.

H. Referensi

Belajarpython.com. (2025). *Tutorial Pemrograman Python Bahasa Indonesia | Dasar Sampai Advance Belajarpython.com*. Belajarpython.com. https://belajarpython.com/

Fahmi, M. N. (2023). *Implementasi Mechine Learning menggunakan Python Library: Scikit-Learn (Supervised dan Unsupervised Learning)*.

Fahmi, N. S., & Asih, M. S. (2025). *Prediksi Produksi Padi Di Pulau Sumatera Menggunakan Metode Ridge Regression*. *2*(2).

Maulana, M. H. (2024). *Python Bahasa Pemograman Yang Ramah Bagi Pemula*. *2*.

PYPL PopularitY of Programming Language index. (2026). https://pypl.github.io/PYPL.html

Suradi, A. A. M., & Rasyid, M. F. (2022). *Sistem Perhitungan Jumlah Kendaraan Berbasis Computer Vision*. (1).